Organización por fases, en el mismo orden que los hitos del enunciado (H0 a H12), y para cada fase indico quién hace qué.

Criterios para repartir

1. **En la App 1, cada uno se encarga de una capa.** Uno se encarga del Backend. El otro hace el Frontend. Así podéis avanzar en paralelo.
2. **En la App 2, intercambiad los papeles.** En la defensa os van a preguntar a cada uno por cualquier parte del flujo, y la rúbrica evalúa la comprensión individual. Si cada uno solo ha tocado su capa, uno de los dos flojeará. Además, hace la comparación entre App 1 y App 2 más interesante. Si preferís no intercambiar, es válido, pero tendréis que explicaros mucho lo que hace cada uno.
3. **Cada uno revisa el código del otro.** Trabajad con ramas y *pull requests*, y que el otro apruebe antes de fusionar. Así queda en Git la "contribución identificable" y la "revisión" que pide el enunciado.
4. **El contrato JSON es el punto de unión.** Mientras esté acordado, cada uno puede trabajar sin esperar al otro.

Desde el primer día, los dos

- **Apuntad cuánto tiempo dedicáis a cada fase**, aunque sea en una nota. Lo necesitaréis para la comparativa de la App 2, y no se puede reconstruir al final.
- Anotad en `docs/proceso.md` las decisiones y los problemas cuando ocurran.
- Cada consulta que hagáis a una IA, apuntadla en `docs/uso-ia.md`: herramienta, para qué, qué aceptasteis y qué rechazasteis.

Fase 0 · Arranque (H0)

Paso	Quién
Cerrar la idea y validarla con el profesor. Preguntadle también qué se debe registrar en la terminal	Los dos
Elegir proveedor y modelo de IA, longitud máxima de entrada y timeout	Los dos
Acordar el contrato: endpoint, campos, tipos, límites, respuesta y errores (200/422/502/504)	Los dos
Crear el repositorio, la estructura de carpetas, `.gitignore` y la protección de `main`	
README inicial (necesidad, miembros, estructura)	
ERS: secciones 1, 2, 6.1 y 7 (introducción, contexto, interfaz, caso de uso) + diagrama de arquitectura	B
ERS: secciones 3, 4, 5 y 6.2 (requisitos funcionales y no funcionales, datos, interfaces técnicas) + diagrama de secuencia	A
ERS: sección 8 (trazabilidad) y 9 (pendientes)	Los dos

Por qué este reparto de la ERS: cada uno redacta lo que mejor conoce. Los requisitos técnicos y los errores los va a implementar el Backend, y la pantalla y la experiencia de usuario el Frontend. La trazabilidad se hace juntos porque une las dos mitades.

Fase 1 · App 1 (H1 a H6)

Backend:

1. **H1**: FastAPI con un endpoint que haga una llamada real a la IA y devuelva algo visible.
2. **H2**: configuración con variables de entorno, modelos Pydantic, límites, timeout, errores controlados, CORS y logging.
3. **H3**: pytest con mocks. Los casos son: válido, vacío, solo espacios, en el límite, por encima del límite, fallo del proveedor, timeout y respuesta inválida.
4. Rellenar `docs/pruebas.md`.

Frontend:

1. **H4**: Vue + Vite, un formulario con etiquetas claras y llamada a `VITE_API_BASE_URL`. Mientras Backend no esté listo, puede trabajar con una respuesta falsa que siga el contrato.
2. Los cuatro estados: entrada, espera, resultado y error. Botón desactivado mientras se espera, para evitar envíos duplicados.
3. **H6**: Tailwind, adaptación a móvil, foco de teclado y comprobar que un texto largo no rompe el diseño.

Juntos:

- **H5 Integración**: probar el flujo completo, incluidos los errores reales del Backend en pantalla.
- **Rotación pequeña**: que uno escriba al menos un test (por ejemplo, el del timeout) y que el otro implement algún detalle del Front (por ejemplo, cómo se muestran los errores 422). Es poco trabajo y os ayudará en la defensa.

Fase 2 · Vercel para la App 1 (H7)

Paso	Quién
Instalar y configurar la CLI de Vercel	Los dos
Proyecto del Backend: raíz `app1/backend`, variables privadas, CORS con la URL real del Front	A
Proyecto del Frontend: raíz `app1/frontend`, `VITE_API_BASE_URL`	B
Probar desde la URL pública un caso válido y otro de error	Los dos
`docs/despliegues.md` y etiqueta Git `app1-final`	A redacta, B revisa

Fase 3 · App 2 con Spec Kit (H8 y H9)

Paso	Quién
Instalar Spec Kit y crear la “constitución”: una pantalla, sin persistencia, una API de IA, mocks…	Los dos
`specify`, `clarify`, `plan`, `tasks` y `analyze`, revisando lo que propone la	Los dos

Paso	Quién
IA	
Commit de la especificación antes de escribir código	Los dos
`docs/ERS-app2.md` (requisitos de Clerk)	B
Implementar el Backend de la App 2 por tareas	**B** (papel invertido)
Implementar el Frontend de la App 2 por tareas	**A** (papel invertido)

Por qué la especificación se hace juntos: es la fase en la que más hay que decidir y corregir a la IA (por ejemplo, si propone una base de datos). En la defensa os pueden preguntar a cualquiera de los dos por esas decisiones.

Fase 4 · Clerk y Vercel para la App 2 (H10)

Paso	Quién
Crear la aplicación en Clerk y configurar el acceso modal en la única pantalla	A (Front de la App 2)
Enviar el token en `Authorization: Bearer`	A
Verificar el token en FastAPI (firma, emisor, caducidad), devolver 401 sin llamar a la IA y escribir sus tests	B (Back de la App 2)
Desplegar en Vercel y probar el acceso aceptado y el rechazado	Los dos

Fase 5 · Docker y EC2 (H11)

Paso	Quién
`Dockerfile.backend` y `.dockerignore`	B
`Dockerfile.frontend` y el proxy (Nginx o Caddy)	A
`compose.yaml` y prueba local desde cero, incluido un reinicio	Los dos
Crear la instancia EC2, el grupo de seguridad, los secretos en el servidor y arrancar	Los dos, en una sesión juntos
HTTPS con dominio, orígenes de Clerk y prueba desde otro dispositivo	Los dos

Por qué juntos en EC2: es la fase con más riesgo (costes, seguridad, red). Trabajando a la vez uno ejecuta y el otro revisa, y se evitan errores como abrir puertos de más o dejar recursos encendidos.

Fase 6 · Cierre (H12)

Paso	Quién
Comparativa: cada uno escribe sobre la capa que hizo en las dos apps	Los dos
README final con instalación, variables, tests y URLs	B
Revisión cruzada de la ERS (sección 10.2): cada uno revisa la parte del otro	Los dos

Paso	Quién
PowerPoint: cada uno prepara las diapositivas de lo que hizo	Los dos
Grabación de respaldo de la demo, por si falla algo externo	A
Ensayar la defensa explicando cada uno la parte del **otro**	Los dos
Detener y eliminar los recursos de AWS cuando el profesor lo indique	A

Cómo trabajar con Git

```
main                        ← solo entra código revisado
 ├── feat/backend-endpoint     (A) → PR → revisa B
 ├── feat/frontend-formulario  (B) → PR → revisa A
 └── docs/ers-inicial          (los dos)
```

- Una rama por tarea y un *pull request* para fusionarla, aprobado por el otro.
- Cuando programéis juntos, añadid `Co-authored-by:` en el commit para que cuente para los dos.
- Etiquetas en los puntos importantes: `app1-final` y `entrega-ud1`.